

3D Gaussian Splatting

Lingjie Liu

3D Gaussian Splatting



Figure 1: Gaussian Splat in Towne 311

Motivation

Ideal 3D Representations:

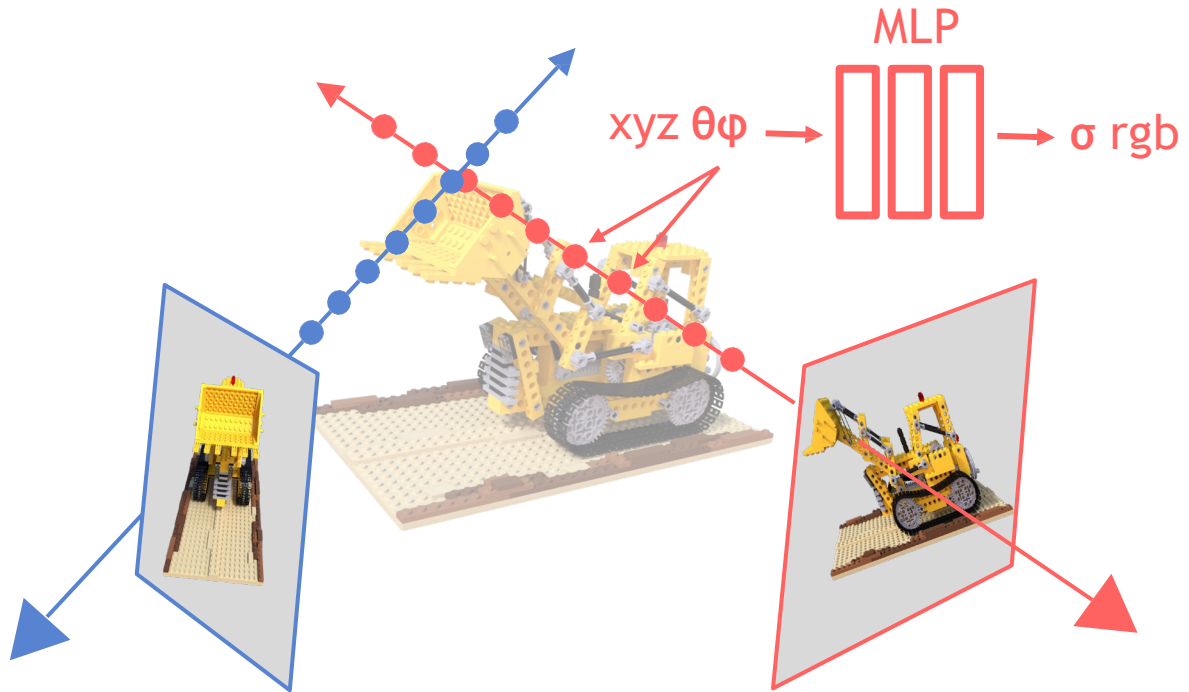
1. Accurate
2. Fast
3. Memory Efficient
4. Practical: easy to integrate in frameworks

Gaussian Splatting:

1. Comparable PSNR with MipNeRF360.
2. 100+ Frames per Second and trains in less than 1h.
3. Renders on Mobile Devices (< 6GB VRAM).
4. Many implementations on different Graphics frameworks.
 - a. Format: easy to standardize (.ply files).

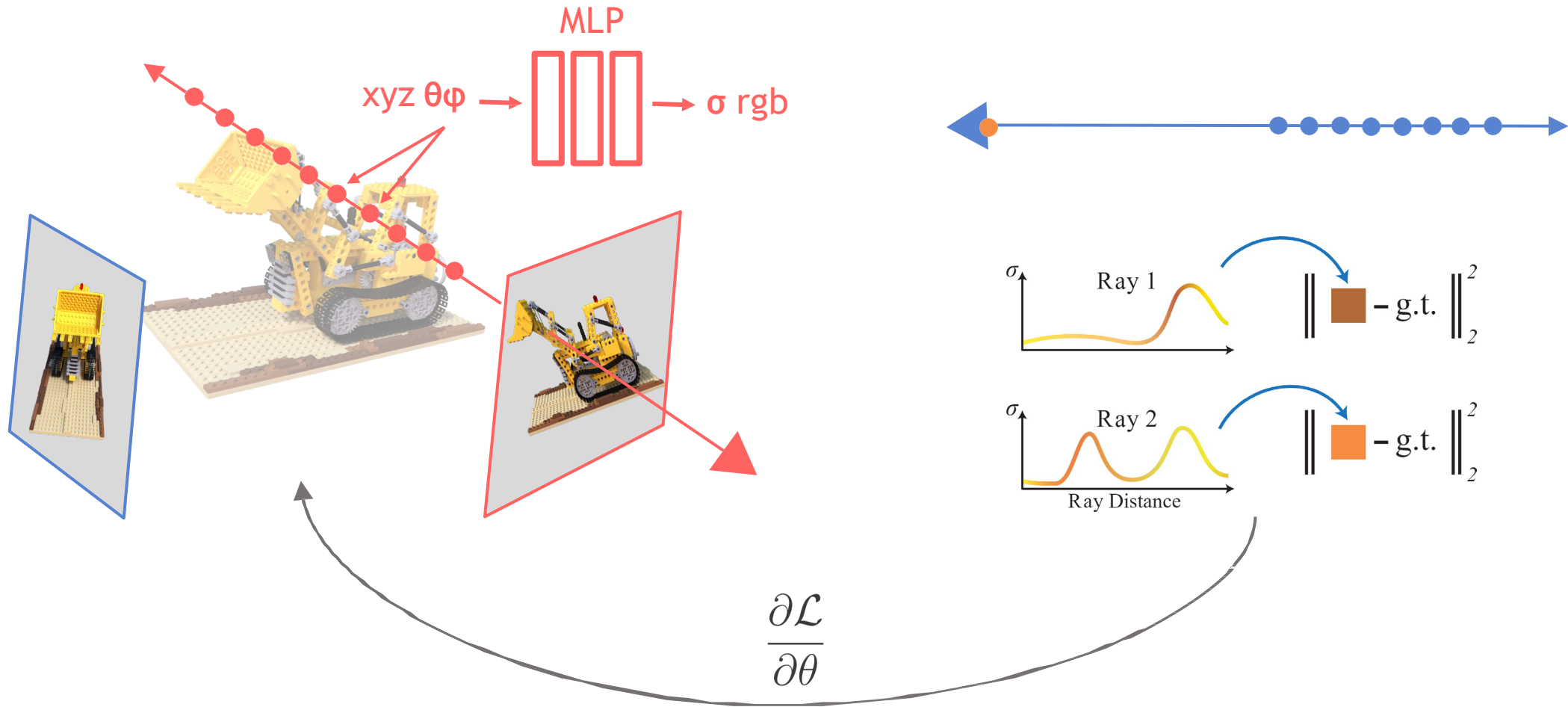
Related Work

Neural Radiance Fields



Related Work

Neural Radiance Fields

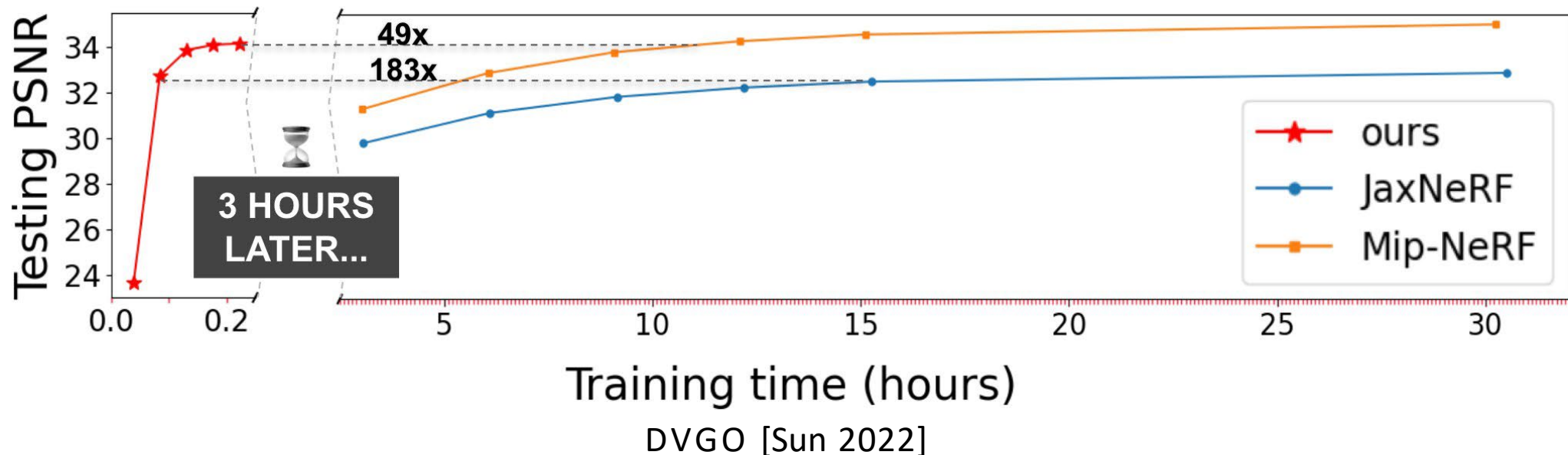


Related Work

Neural Radiance Fields

NeRF suffers from slow training and rendering

- NSVF [Liu 2019]
- DVGO [Sun 2022]
- Instant-NGP [Müller 2022]
- Plenoxels [Fridovich-Keil and Yu 2022]
- TensorRF [Chen and Xu 2022]

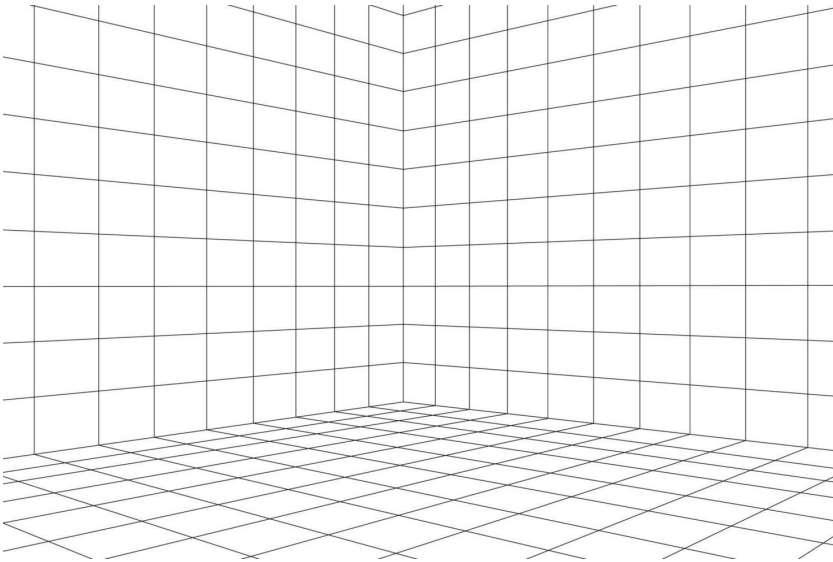


Related Work

Point-Based Representations

Eulerian (NeRFs)

- Queries in 3D Space



Lagrangian

- Access to primitives



Background

Traditional Point-Based Graphics

Surface Splatting - Zwicker et al. 2001

(EWA – Elliptical Weighted Average)

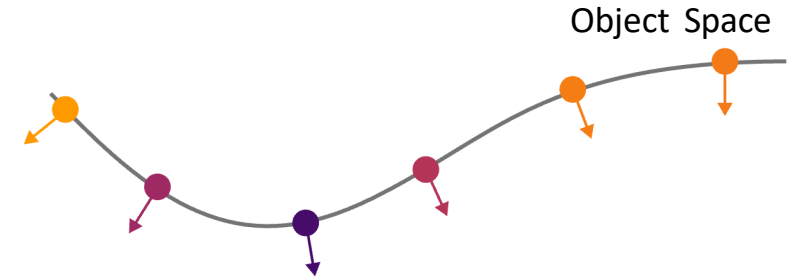
Background

Traditional Point-Based Graphics

Surface Splatting - Zwicker et al. 2001

(EWA – Elliptical Weighted Average)

1. Considers oriented points (surfels) as discrete samples of a texture function on a surface.



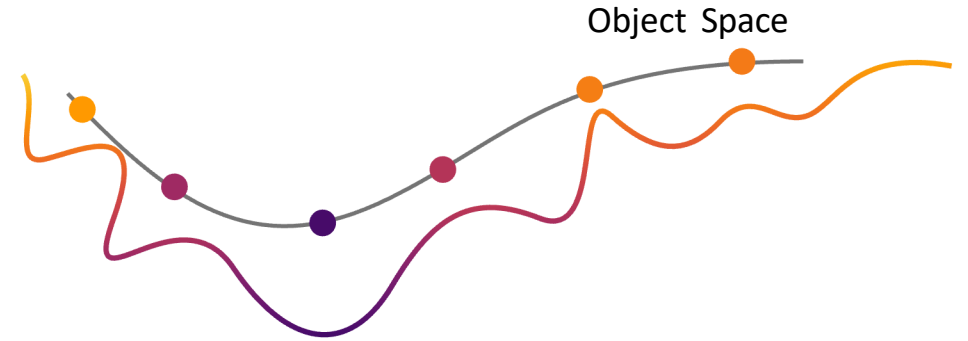
Background

Traditional Point-Based Graphics

Surface Splatting - Zwicker et al. 2001

(EWA – Elliptical Weighted Average)

1. Considers oriented points (surfels) as discrete samples of a texture function on a surface.
2. A Gaussian reconstruction kernel is used to recover a continuous signal.



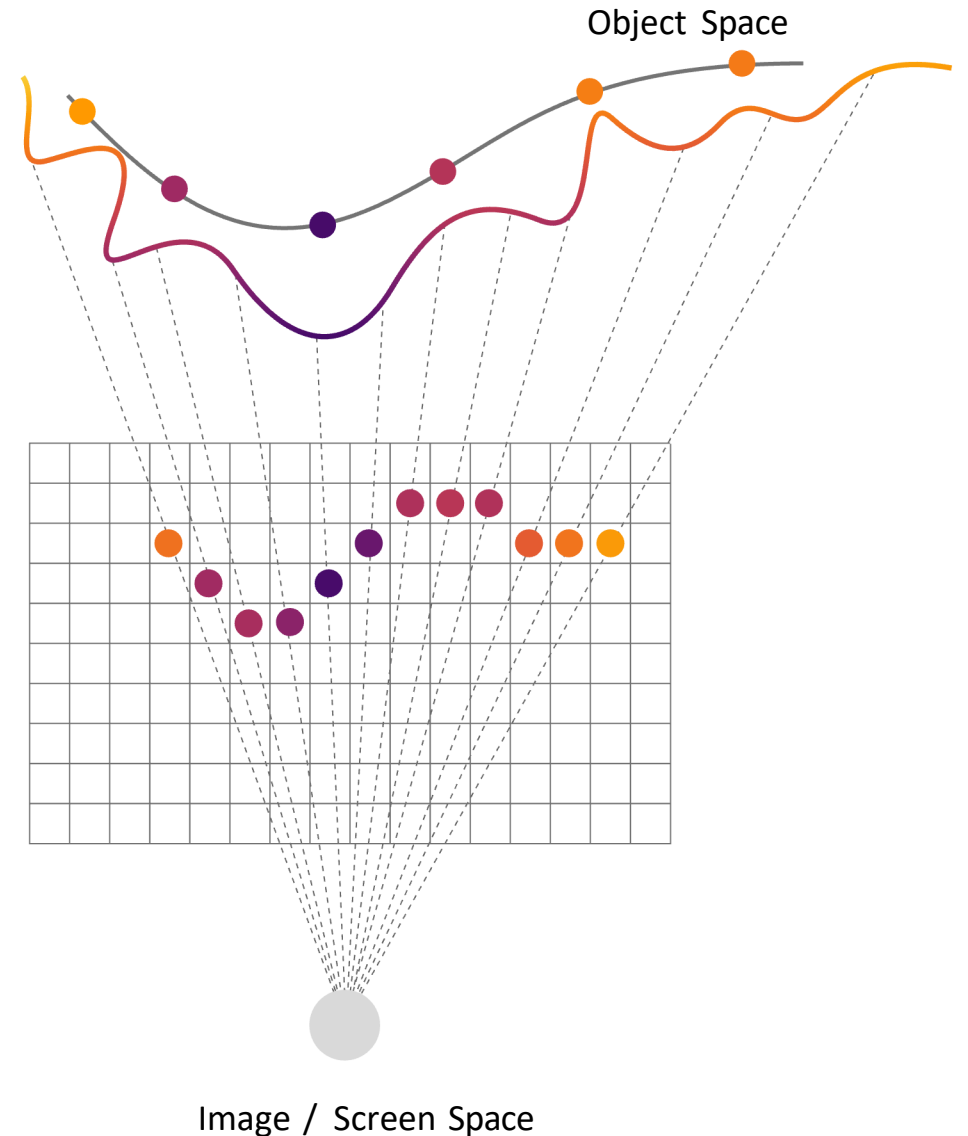
Background

Traditional Point-Based Graphics

Surface Splatting - Zwicker et al. 2001

(EWA – Elliptical Weighted Average)

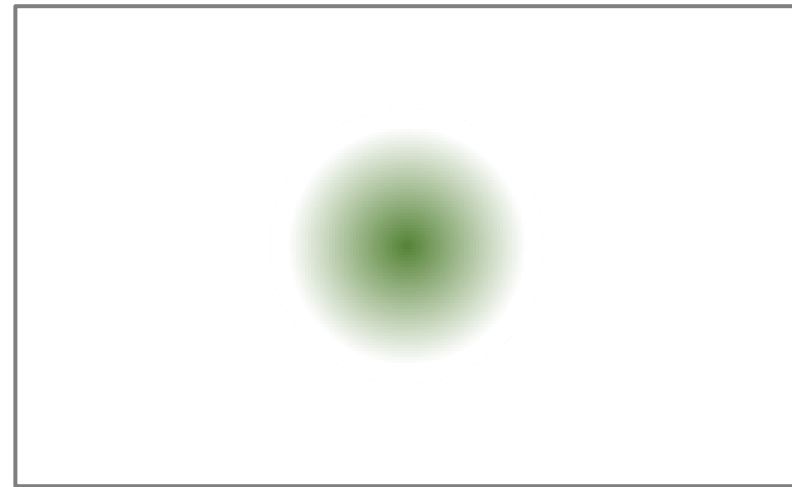
1. Considers oriented points (surfels) as discrete samples of a texture function on a surface.
2. A Gaussian reconstruction kernel is used to recover a continuous signal.
3. Such that we can sample it in screen space.



Background

Traditional Point-Based Graphics

The important outcomes of this algorithm are:



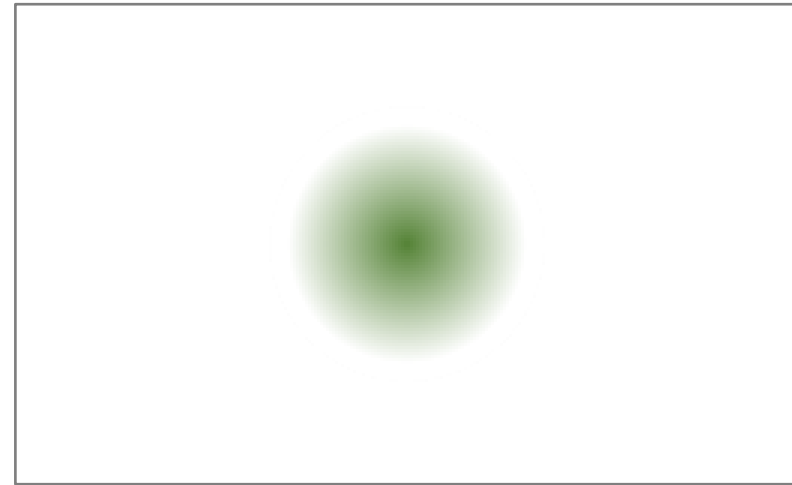
Image

Background

Traditional Point-Based Graphics

The important outcomes of this algorithm are:

1. Moving camera closer, scales the points so the objects have no holes.



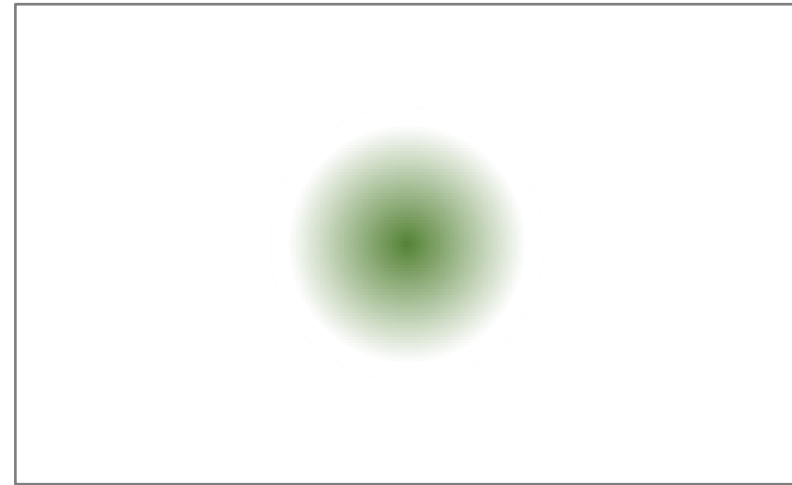
Image

Background

Traditional Point-Based Graphics

The important outcomes of this algorithm are:

1. Moving camera closer, scales the points so the objects have no holes.
2. Slanted normals appear as ellipses, so we can create better edges.



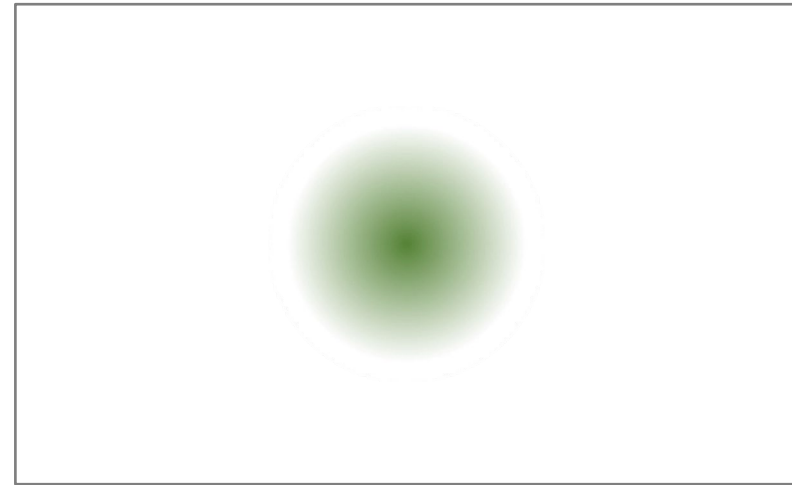
Image

Background

Traditional Point-Based Graphics

The important outcomes of this algorithm are:

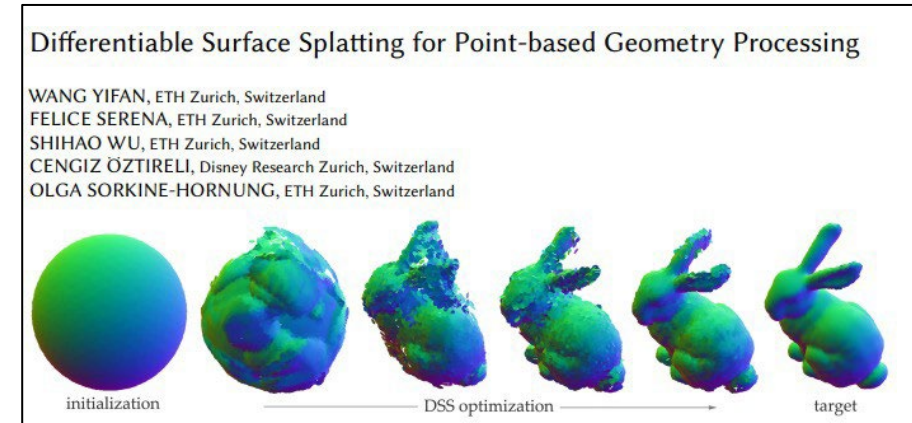
1. Moving camera closer, scales the points so the objects have no holes.
2. Slanted normals appear as ellipses, so we can create better edges.
3. Each sample can be processed independently



Image

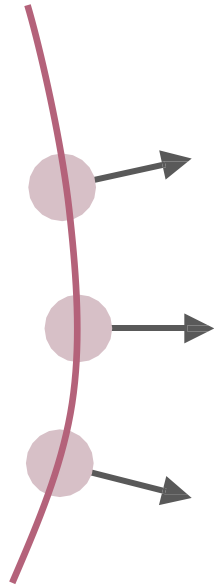
Background

Recent Advances in Point Clouds



- Differentiable Surface Splatting [Yifan '19] showed that this process is end-to-end differentiable.
- 3DGS is heavily inspired and builds on top of this line of work.

3D Gaussian Spats



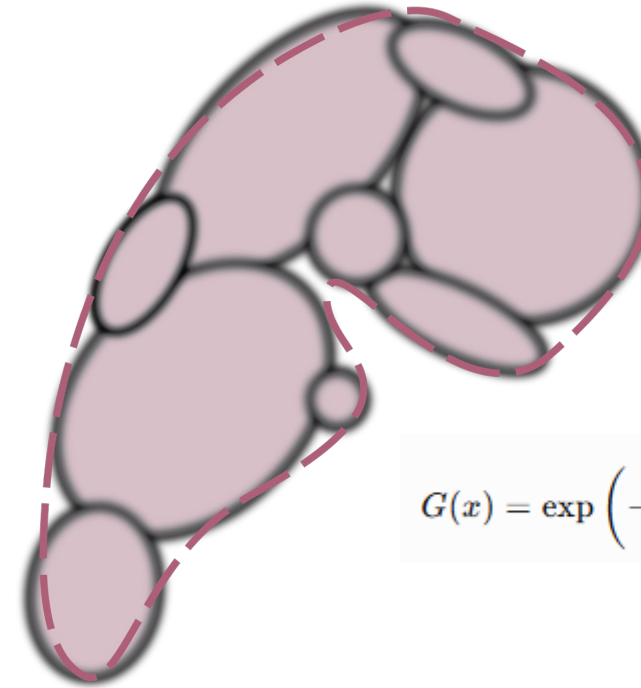
position

normal

$$p = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

$$n = \begin{bmatrix} n_x \\ n_y \\ n_z \end{bmatrix}$$

σ std dev
 f appearance



$$G(x) = \exp \left(-\frac{1}{2} (x - p)^T \Sigma^{-1} (x - p) \right)$$

$$p = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} \sigma_x & \sigma_{xy} & \sigma_{xz} \\ & \sigma_y & \sigma_{yz} \\ & & \sigma_z \end{bmatrix}$$

covariance matrix

o SH spherical harmonics

3D Gaussian Splats

A 3D gaussian is defined by:

- **Position (Mean μ):** location (XYZ)
- **Covariance Matrix (Σ):** rotation and scaling
- **Opacity (α):** Transparency
- **Color (RGB) or Spherical Harmonics (SH)** coefficients

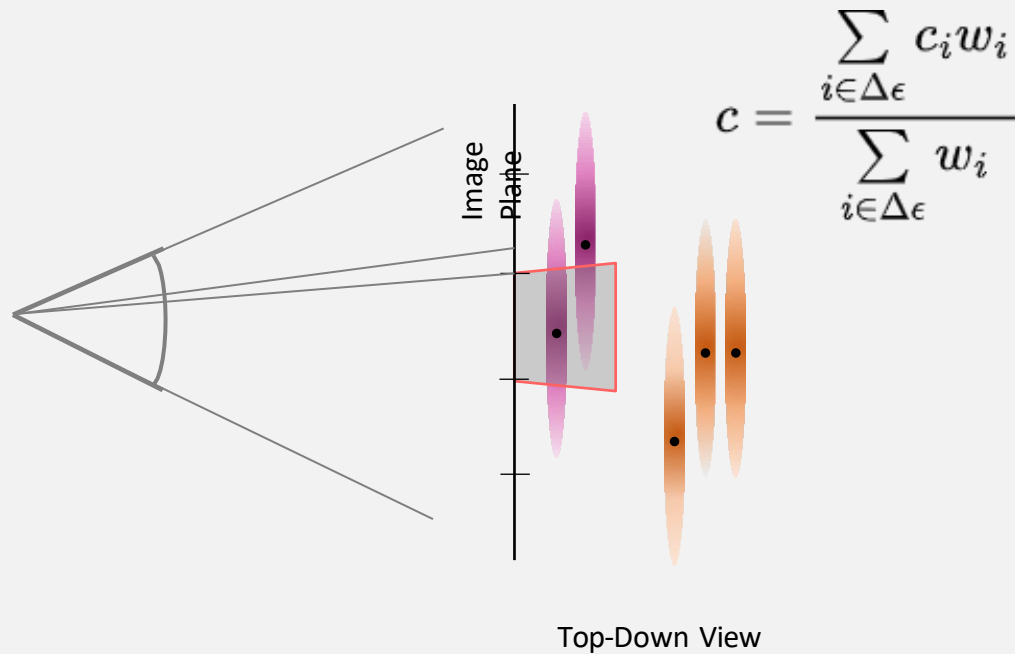
Surface Splatting vs Volume Splatting

1. How do we blend points in screen space?

Surface Splatting vs Volume Splatting

1. How do we blend points in screen space?

[Zwicker1 '01] / [Yifan '19]



Surface Splatting vs Volume

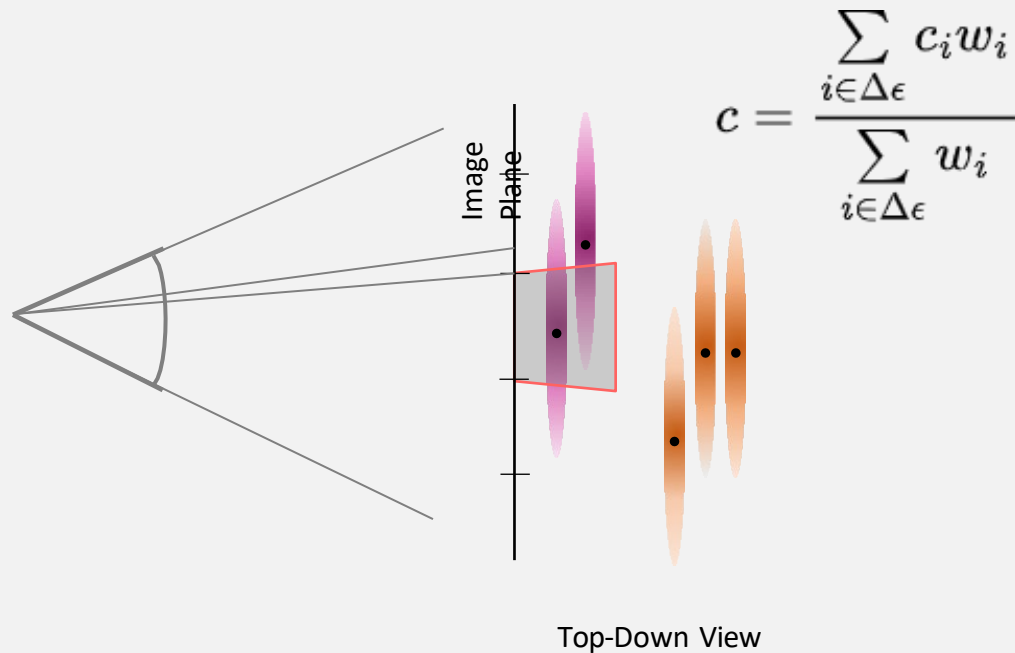
1. How do we blend points in screen space?

Sampling enables us to represent a continuous scene representation because it results in the MLP being evaluated at continuous positions over the course of optimization. We use these samples to estimate $C(\mathbf{r})$ with the quadrature rule discussed in the volume rendering review by Max [26]:

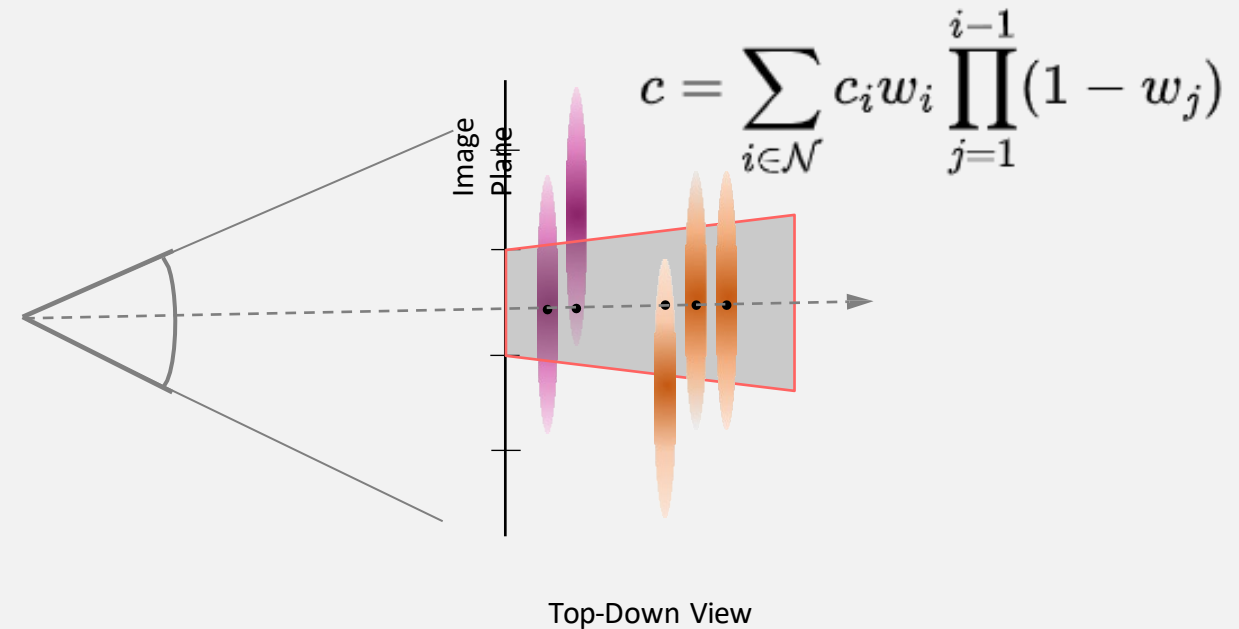
$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) \mathbf{c}_i, \text{ where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right), \quad (3)$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples. This function for calculating $\hat{C}(\mathbf{r})$ from the set of $(\mathbf{c}_i, \sigma_i)$ values is trivially differentiable and reduces to traditional alpha compositing with alpha values $\alpha_i = 1 - \exp(-\sigma_i \delta_i)$.

[Zwicker1 '01] / [Yifan '19]



[Zwicker2 '01] / [Kerbl & Kopanas '23]



[Zwicker1 '01] Surface Splatting

[Yifan '19] Differentiable Surface Splatting for Point-Based Geometry Processing

[Zwicker2 '01] EWA Volume Splatting

[Kerbl & Kopanas '23] 3D Gaussian Splatting for Real-Time Radiance Field Rendering

3D Gaussian Splatting Overview

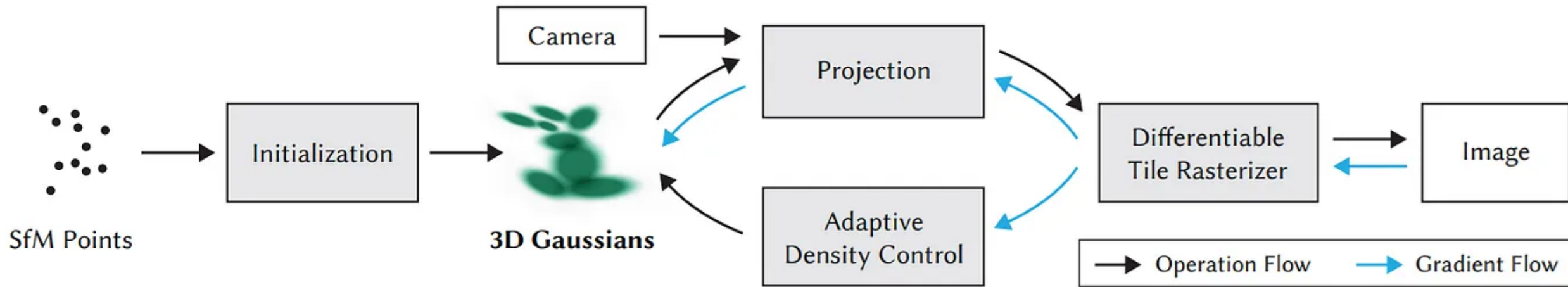


Figure 2: Starting from a sparse Structure-from-Motion (SfM) point cloud, the optimization process used a fast tile-based renderer and generates a set of 3D Gaussians, and their density is adaptively controlled.

Training Process

- Structure from Motion (SfM) initialization
- Gradient Descent for Parameter Optimization
- Adaptive Densification

Training Process

Algorithm 1 Optimization and Densification

w, h : width and height of the training images

```

 $M \leftarrow$  SfM Points ▷ Positions
 $S, C, A \leftarrow$  InitAttributes() ▷ Covariances, Colors, Opacities
 $i \leftarrow 0$  ▷ Iteration Count

```

while not converged **do**

```

 $V, \hat{I} \leftarrow$  SampleTrainingView() ▷ Camera  $V$  and Image
 $I \leftarrow$  Rasterize( $M, S, C, A, V$ ) ▷ Alg. 2
 $L \leftarrow$  Loss( $I, \hat{I}$ ) ▷ Loss
 $M, S, C, A \leftarrow$  Adam( $\nabla L$ ) ▷ Backprop & Step

```

if IsRefinementIteration(i) **then**

```

for all Gaussians ( $\mu, \Sigma, c, \alpha$ ) in ( $M, S, C, A$ ) do
    if  $\alpha < \epsilon$  or IsTooLarge( $\mu, \Sigma$ ) then ▷ Pruning
        RemoveGaussian()
    end if
    if  $\nabla_p L > \tau_p$  then ▷ Densification
        if  $\|S\| > \tau_S$  then ▷ Over-reconstruction
            SplitGaussian( $\mu, \Sigma, c, \alpha$ )
        else ▷ Under-reconstruction
            CloneGaussian( $\mu, \Sigma, c, \alpha$ )
        end if
    end if
end for

```

end if

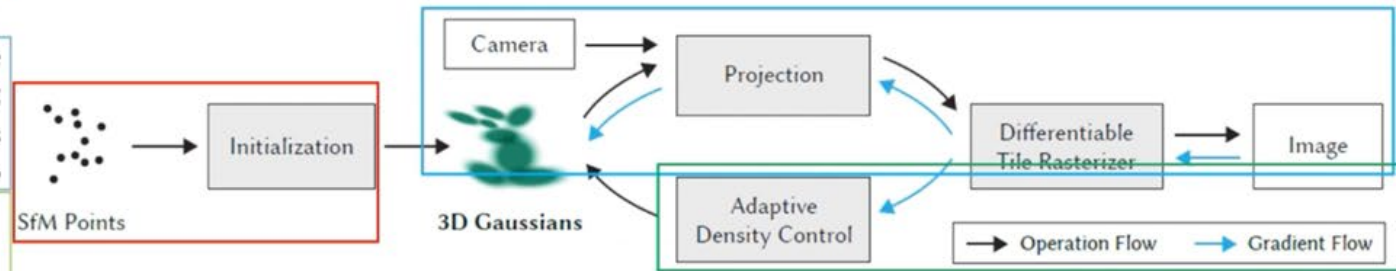
end if

end for

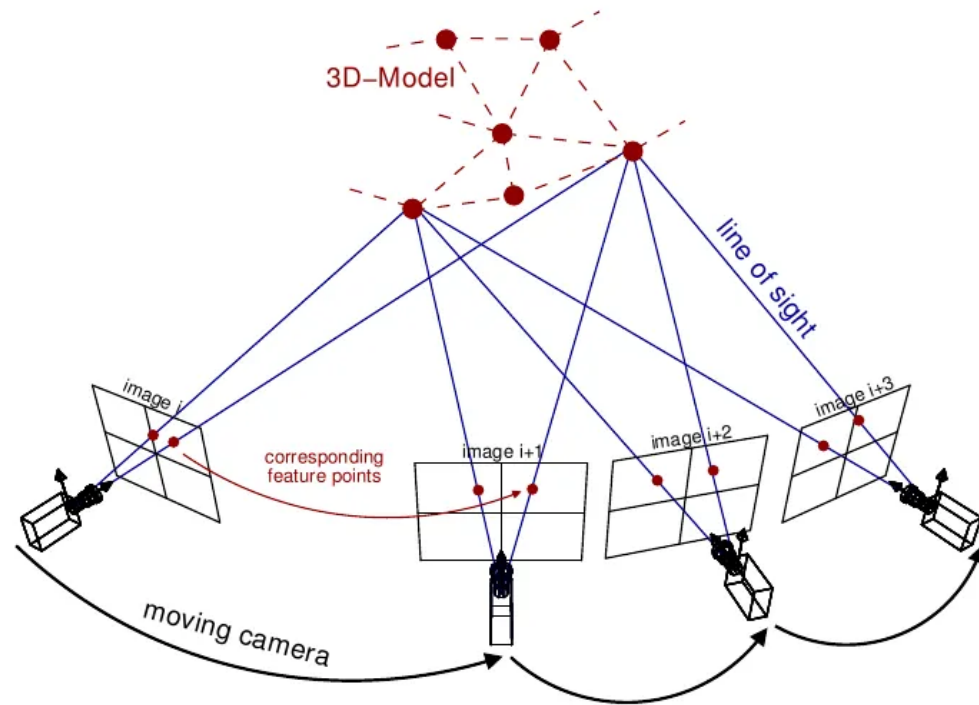
end if

$i \leftarrow i + 1$

end while



1st step: Structure from Motion



2nd step: Initialization

M : the center point of initial gaussian = mean of gaussian



Algorithm 1 Optimization and Densification

w, h : width and height of the training images

$M \leftarrow \text{SfM Points}$ ▷ Positions
 $S, C, A \leftarrow \text{InitAttributes}()$ ▷ Covariances, Colors, Opacities
 $i \leftarrow 0$ ▷ Iteration Count

while not converged **do**

$V, \hat{I} \leftarrow \text{SampleTrainingView}()$ ▷ Camera V and Image

$I \leftarrow \text{Rasterize}(M, S, C, A, V)$ ▷ Alg. 2

$L \leftarrow \text{Loss}(I, \hat{I})$ ▷ Loss

$M, S, C, A \leftarrow \text{Adam}(\nabla L)$ ▷ Backprop & Step

if IsRefinementIteration(i) **then**

for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**

if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning

RemoveGaussian()

end if

if $\nabla_p L > \tau_p$ **then** ▷ Densification

if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction

SplitGaussian(μ, Σ, c, α)

else ▷ Under-reconstruction

CloneGaussian(μ, Σ, c, α)

end if

end if

end for

end if

$i \leftarrow i + 1$

end while

2nd step: Initialization

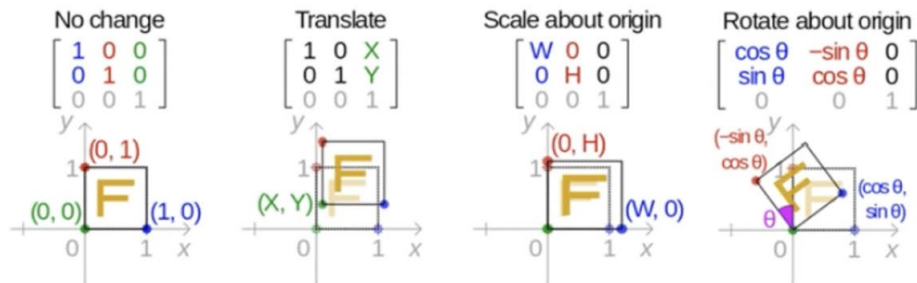
$\underline{S}$: covariance of gaussian = symmetric matrix

$$\text{var}(\mathbf{X}) = \mathbb{E}[(\mathbf{X} - \mathbb{E}[\mathbf{X}])(\mathbf{X} - \mathbb{E}[\mathbf{X}])^T]$$

Σ (covariance)

$$\Sigma = R S S^T R^T$$

R(rotation matrix) : 4x1 quaternion vector to 3x3 rotation matrix
S(scale matrix) : 3x1 scale vector to 3x3 scale matrix



Algorithm 1 Optimization and Densification

w, h : width and height of the training images

```

M ← SfM Points                                ▷ Positions
S, C, A ← InitAttributes()                    ▷ Covariances, Colors, Opacities
i ← 0                                          ▷ Iteration Count

```

while not converged **do**

$V, \hat{I} \leftarrow \text{SampleTrainingView}()$ ▷ Camera V and Image

$I \leftarrow \text{Rasterize}(M, S, C, A, V)$ ▷ Alg. 2

$L \leftarrow \text{Loss}(I, \hat{I})$ ▷ Loss

$M, S, C, A \leftarrow \text{Adam}(\nabla L)$ ▷ Backprop & Step

if IsRefinementIteration(i) **then**

for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**

if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning

RemoveGaussian()

end if

if $\nabla_p L > \tau_p$ **then** ▷ Densification

if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction

SplitGaussian(μ, Σ, c, α)

else ▷ Under-reconstruction

CloneGaussian(μ, Σ, c, α)

end if

end if

end for

end if

$i \leftarrow i + 1$

end while

2nd step: Initialization

$\underline{C}$: color of gaussian = represented by spherical harmonic functions

$$C = \sum_{l=0}^{l_{max}} \sum_{m=-l}^l k_l^m Y_l^m(\theta, \phi)$$

Algorithm 1 Optimization and Densification

w, h : width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions
 $S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
 $i \leftarrow 0$ ▷ Iteration Count

while not converged **do**

$V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image

$I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2

$L \leftarrow$ Loss($I, \hat{I}$) ▷ Loss

$M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step

if IsRefinementIteration(i) **then**

for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**

if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning

RemoveGaussian()

end if

if $\nabla_p L > \tau_p$ **then** ▷ Densification

if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction

SplitGaussian(μ, Σ, c, α)

else ▷ Under-reconstruction

CloneGaussian(μ, Σ, c, α)

end if

end if

end for

end if

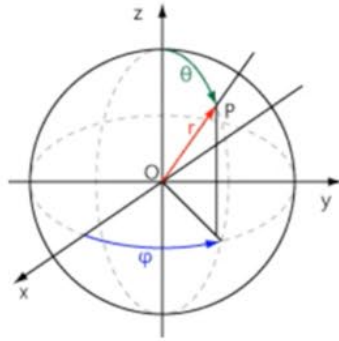
$i \leftarrow i + 1$

end while

2nd step: Initialization

Spherical Harmonics(SH)

: functions defined on the surface of a sphere.



Spherical coordinate system

$$Y_l^m(\theta, \phi) = \sqrt{\frac{(2l+1)(l-|m|)!}{4\pi(l+|m|)!}} P_l^{|m|}(\cos \theta) e^{im\phi}$$

$$P_\ell^{(|m|)}(\cos \theta) = (-1)^m \frac{(\ell+|m|)!}{(\ell-|m|)!} P_\ell^{(-|m|)}(\cos \theta)$$

$$C = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^l k_l^m Y_l^m(\theta, \phi) = k_0^0 * Y_0^0(\theta, \phi) + k_1^{-1} * Y_1^{-1}(\theta, \phi) + k_1^0 * Y_1^0(\theta, \phi) + k_1^1 * Y_1^1(\theta, \phi) + \dots + k_2^1 * Y_2^1(\theta, \phi) + k_2^2 * Y_2^2(\theta, \phi)$$

Made by
XO

$$Y_0^0(\theta, \varphi) = \frac{1}{2} \sqrt{\frac{1}{\pi}}$$

$$Y_1^{-1}(\theta, \varphi) = \frac{1}{2} \sqrt{\frac{3}{2\pi}} \cdot e^{-i\varphi} \cdot \sin \theta$$

$$Y_1^0(\theta, \varphi) = \frac{1}{2} \sqrt{\frac{3}{\pi}} \cdot \cos \theta$$

$$Y_1^1(\theta, \varphi) = -\frac{1}{2} \sqrt{\frac{3}{2\pi}} \cdot e^{i\varphi} \cdot \sin \theta$$

$$Y_2^{-2}(\theta, \varphi) = \frac{1}{4} \sqrt{\frac{15}{2\pi}} \cdot e^{-2i\varphi} \cdot \sin^2 \theta$$

$$Y_2^{-1}(\theta, \varphi) = \frac{1}{2} \sqrt{\frac{15}{2\pi}} \cdot e^{-i\varphi} \cdot \sin \theta \cdot \cos \theta$$

$$Y_2^0(\theta, \varphi) = \frac{1}{4} \sqrt{\frac{5}{\pi}} \cdot (3 \cos^2 \theta - 1)$$

$$Y_2^1(\theta, \varphi) = -\frac{1}{2} \sqrt{\frac{15}{2\pi}} \cdot e^{i\varphi} \cdot \sin \theta \cdot \cos \theta$$

$$Y_2^2(\theta, \varphi) = \frac{1}{4} \sqrt{\frac{15}{2\pi}} \cdot e^{2i\varphi} \cdot \sin^2 \theta$$

2nd step: Initialization

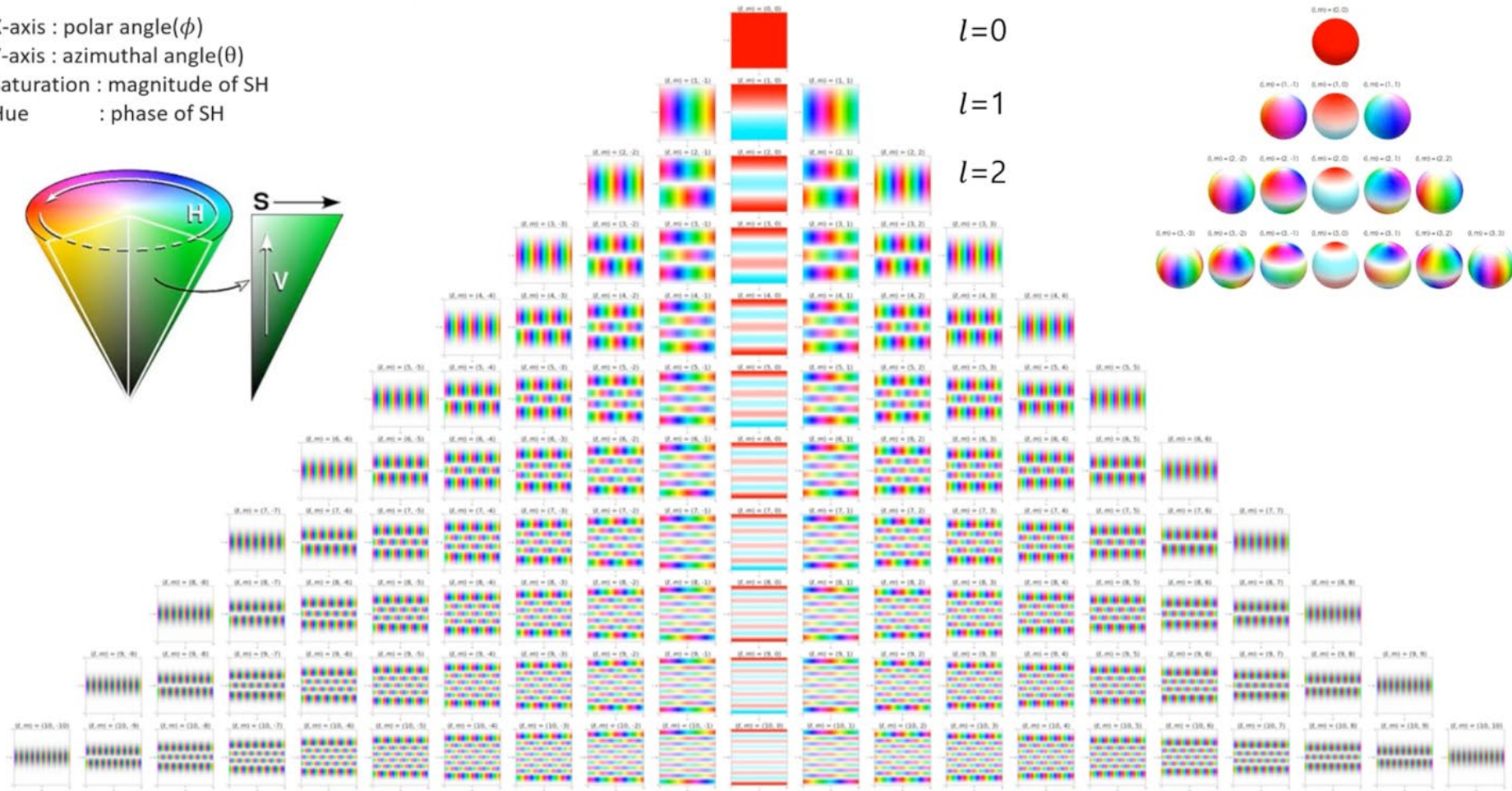
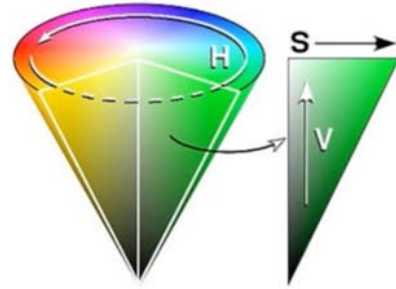
Spherical Harmonics – Texture Map

X-axis : polar angle(ϕ)

Y-axis : azimuthal angle(θ)

Saturation : magnitude of SH

Hue : phase of SH



2nd step: Initialization

A : alpha of gaussian = opacity of gaussian = a real number

Algorithm 1 Optimization and Densification

w, h : width and height of the training images

$M \leftarrow \text{SfM Points}$ ▷ Positions
 $S, C, A \leftarrow \text{InitAttributes}()$ ▷ Covariances, Colors, Opacities
 $i \leftarrow 0$ ▷ Iteration Count

while not converged **do**

$V, \hat{I} \leftarrow \text{SampleTrainingView}()$ ▷ Camera V and Image

$I \leftarrow \text{Rasterize}(M, S, C, A, V)$ ▷ Alg. 2

$L \leftarrow \text{Loss}(I, \hat{I})$ ▷ Loss

$M, S, C, A \leftarrow \text{Adam}(\nabla L)$ ▷ Backprop & Step

if IsRefinementIteration(i) **then**

for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**

if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning

RemoveGaussian()

end if

if $\nabla_p L > \tau_p$ **then** ▷ Densification

if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction

SplitGaussian(μ, Σ, c, α)

else ▷ Under-reconstruction

CloneGaussian(μ, Σ, c, α)

end if

end if

end for

end if

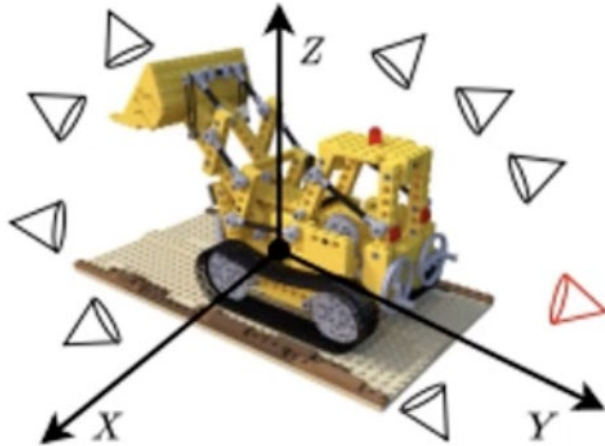
$i \leftarrow i + 1$

end while

2nd step: Optimization

Sample Training View : read input data

- Input values : V (View Direction), I' (GT 2D image)



Algorithm 1 Optimization and Densification

w, h : width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions
 $S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
 $i \leftarrow 0$ ▷ Iteration Count

while not converged **do**

$V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image
 $I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2
 $L \leftarrow$ Loss($I, \hat{I}$) ▷ Loss
 $M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step

if IsRefinementIteration(i) **then**

for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**

if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning
RemoveGaussian()
end if

if $\nabla_p L > \tau_p$ **then** ▷ Densification
if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction
SplitGaussian(μ, Σ, c, α)
else ▷ Under-reconstruction
CloneGaussian(μ, Σ, c, α)
end if

end if
end for

end if
end for

$i \leftarrow i + 1$

end while

2nd step: Optimization

Algorithm 1 Optimization and Densification

w, h : width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions

$S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities

$i \leftarrow 0$ ▷ Iteration Count

while not converged **do**

$V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image

$I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2

$L \leftarrow$ Loss($I, \hat{I}$) ▷ Loss

$M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step

if IsRefinementIteration(i) **then**

for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**

if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning

RemoveGaussian()

end if

if $\nabla_p L > \tau_p$ **then** ▷ Densification

if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction

SplitGaussian(μ, Σ, c, α)

else ▷ Under-reconstruction

CloneGaussian(μ, Σ, c, α)

end if

end if

end for

end if

$i \leftarrow i + 1$

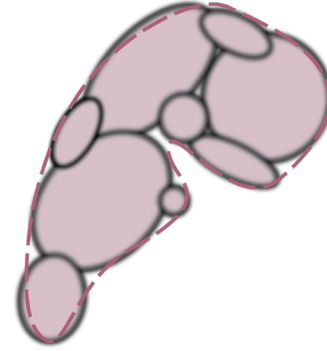
end while

Rasterize : using tile-based rasterizer

- Input values : M (Mean=xyz), S (Covariance), C (Color), A (Opacity), V (View Direction)
- Output values : I (2D Image)

How do we render?

1. Sort: globally based on depth
2. Splat: compute the shape of the Gaussian after projection
3. Blend: alpha composite



$$p = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \quad o \quad SH$$

$$\Sigma = \begin{bmatrix} \sigma_x & \sigma_{xy} & \sigma_{xz} \\ & \sigma_y & \sigma_{yz} \\ & & \sigma_z \end{bmatrix}$$

How did we go from 5 FPS to 100+ FPS?

and from 18h to 40min for training

Using the GPU efficiently:

1. Tiling

Split the image in 16x16 Tiles – helps threads to work collaboratively.

2. Single global sort

GPU sorts millions of primitives fast.

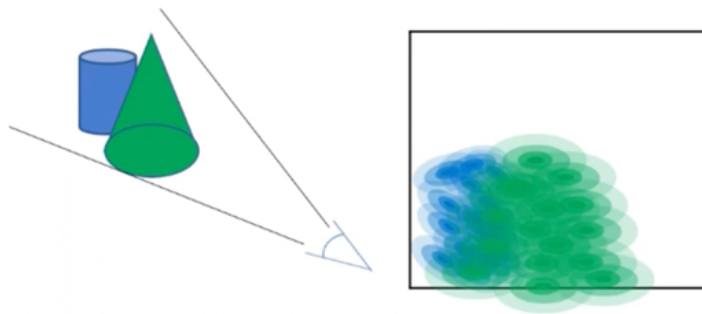
Fast Differentiable Rasterizer

Tile-based Rasterizer

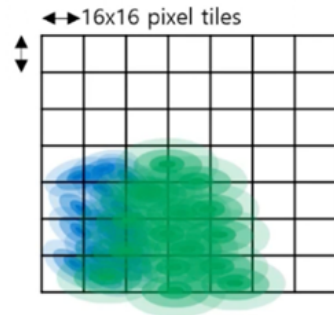
- 1) Cull Gaussian : Frustum Culling
- 2) Screen Space Gaussian : 3D to 2D Projection

Covariance Projection function

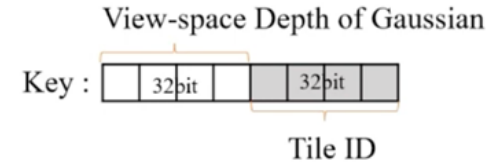
$$\Sigma' = JW \Sigma W^T J^T$$



- 3) Create Tiles : split the screen into 16x16 pixel tiles



- 4) Duplicate with Keys : generate keys of each gaussian every tile



- 5) Sorted by Keys : depth ordering by Radix Sort each tile
 - 6) Identify Tile Ranges : identify Gaussian list each tile
 - 7) Get Tile Range : read Gaussian list
-> to maximize parallelism for loading/sharing and forward/backward processing
 - 8) Blend in Order
: Accumulates color and α values by traversing the lists front-to-back each pixel.
: When a target saturation of α in a pixel, the corresponding thread stops.
- +) backward process
: propagate gradients following the ratio of the opacity of gaussian

2nd step: Optimization

Algorithm 1 Optimization and Densification

w, h : width and height of the training images

$M \leftarrow \text{SfM Points}$ ▷ Positions
 $S, C, A \leftarrow \text{InitAttributes}()$ ▷ Covariances, Colors, Opacities
 $i \leftarrow 0$ ▷ Iteration Count

while not converged **do**

$V, \hat{I} \leftarrow \text{SampleTrainingView}()$ ▷ Camera V and Image
 $I \leftarrow \text{Rasterize}(M, S, C, A, V)$ ▷ Alg. 2
 $L \leftarrow \text{Loss}(I, \hat{I})$ ▷ Loss
 $M, S, C, A \leftarrow \text{Adam}(\nabla L)$ ▷ Backprop & Step

if IsRefinementIteration(i) **then**

for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**

if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning
 RemoveGaussian()
end if

if $\nabla_p L > \tau_p$ **then** ▷ Densification
 if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction
 SplitGaussian(μ, Σ, c, α)
 else ▷ Under-reconstruction
 CloneGaussian(μ, Σ, c, α)
 end if

end if

end if

end for

end if

$i \leftarrow i + 1$

end while

Loss : measure differences between GT and inference image

$$\mathcal{L} = (1 - \lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}}$$

Loss function is calculated by combining L1 and D-SSIM (Structural Dissimilarity Index)

2nd step: Optimization

Algorithm 1 Optimization and Densification

w, h : width and height of the training images

$M \leftarrow \text{SfM Points}$ ▷ Positions
 $S, C, A \leftarrow \text{InitAttributes}()$ ▷ Covariances, Colors, Opacities
 $i \leftarrow 0$ ▷ Iteration Count

while not converged **do**

$V, \hat{I} \leftarrow \text{SampleTrainingView}()$ ▷ Camera V and Image
 $I \leftarrow \text{Rasterize}(M, S, C, A, V)$ ▷ Alg. 2
 $L \leftarrow \text{Loss}(I, \hat{I})$ ▷ Loss
 $M, S, C, A \leftarrow \text{Adam}(\nabla L)$ ▷ Backprop & Step

if IsRefinementIteration(i) **then**

for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**

if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning
 RemoveGaussian()
end if

if $\nabla_p L > \tau_p$ **then** ▷ Densification
 if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction
 SplitGaussian(μ, Σ, c, α)
 else ▷ Under-reconstruction
 CloneGaussian(μ, Σ, c, α)
 end if

end if
end if

end for

end if

$i \leftarrow i + 1$

end while

Adam : backpropagation into variables

- Updated variables : M(Mean=xyz), S(Covariance), C(Color), A(Opacity)

3rd step: Adaptive Densification

After every 100
iterations: Densify!

Algorithm 1 Optimization and Densification

w, h : width and height of the training images

$M \leftarrow$ SfM Points ▷ Positions
 $S, C, A \leftarrow$ InitAttributes() ▷ Covariances, Colors, Opacities
 $i \leftarrow 0$ ▷ Iteration Count

while not converged **do**

$V, \hat{I} \leftarrow$ SampleTrainingView() ▷ Camera V and Image

$I \leftarrow$ Rasterize(M, S, C, A, V) ▷ Alg. 2

$L \leftarrow$ Loss($I, \hat{I}$) ▷ Loss

$M, S, C, A \leftarrow$ Adam(∇L) ▷ Backprop & Step

if IsRefinementIteration(i) **then**

for all Gaussians (μ, Σ, c, α) **in** (M, S, C, A) **do**

if $\alpha < \epsilon$ or IsTooLarge(μ, Σ) **then** ▷ Pruning

RemoveGaussian()

end if

if $\nabla_p L > \tau_p$ **then** ▷ Densification

if $\|S\| > \tau_S$ **then** ▷ Over-reconstruction

SplitGaussian(μ, Σ, c, α)

else ▷ Under-reconstruction

CloneGaussian(μ, Σ, c, α)

end if

end if

end for

end if

$i \leftarrow i + 1$

end while

3rd step: Adaptive Densification

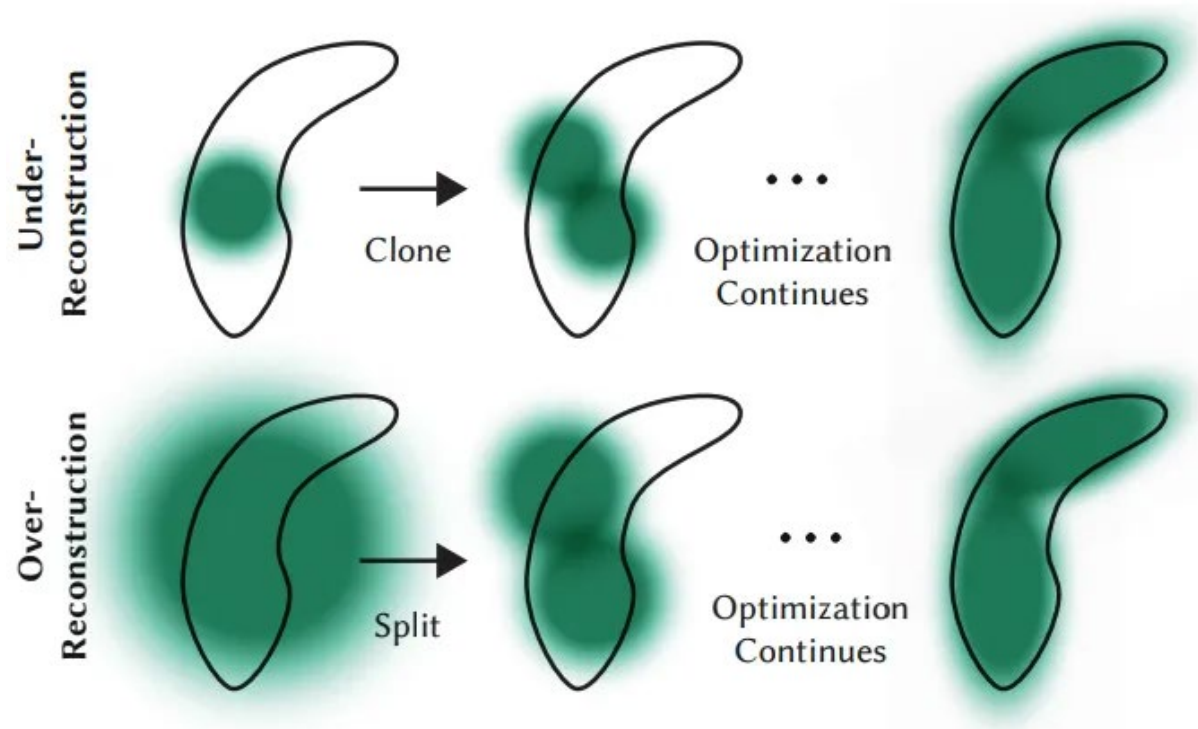


Figure 4: Top row (under reconstruction) and Bottom row (over-reconstruction)

Results and Analysis



A side-by-side comparison of previous high-quality representations and Gaussian Splatting (marked as “Ours”) in terms of rendering speed (fps), training time (min), and visual quality (Peak signal-to-noise ratio, the higher the better)

Results and Analysis

Dataset Method Metric	Mip-NeRF360						Tanks&Temples						Deep Blending					
	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem
Plenoxels	0.626	23.08	0.463	25m49s	6.79	2.1GB	0.719	21.08	0.379	25m5s	13.0	2.3GB	0.795	23.06	0.510	27m49s	11.2	2.7GB
INGP-Base	0.671	25.30	0.371	5m37s	11.7	13MB	0.723	21.72	0.330	5m26s	17.1	13MB	0.797	23.62	0.423	6m31s	3.26	13MB
INGP-Big	0.699	25.59	0.331	7m30s	9.43	48MB	0.745	21.92	0.305	6m59s	14.4	48MB	0.817	24.96	0.390	8m	2.79	48MB
M-NeRF360	0.792 [†]	27.69 [†]	0.237 [†]	48h	0.06	8.6MB	0.759	22.22	0.257	48h	0.14	8.6MB	0.901	29.40	0.245	48h	0.09	8.6MB
Ours-7K	0.770	25.60	0.279	6m25s	160	523MB	0.767	21.20	0.280	6m55s	197	270MB	0.875	27.78	0.317	4m35s	172	386MB
Ours-30K	0.815	27.21	0.214	41m33s	134	734MB	0.841	23.14	0.183	26m54s	154	411MB	0.903	29.41	0.243	36m2s	137	676MB

Very fast!!!! Reached 197 FPS with similar losses among other methods.

Interactive Results

<https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>

or

Google search: “3D Gaussian Splatting”



Ours



MipNeRF360

Ablation Study - Anisotropy



Ground Truth



Full



Isotropic

Applications

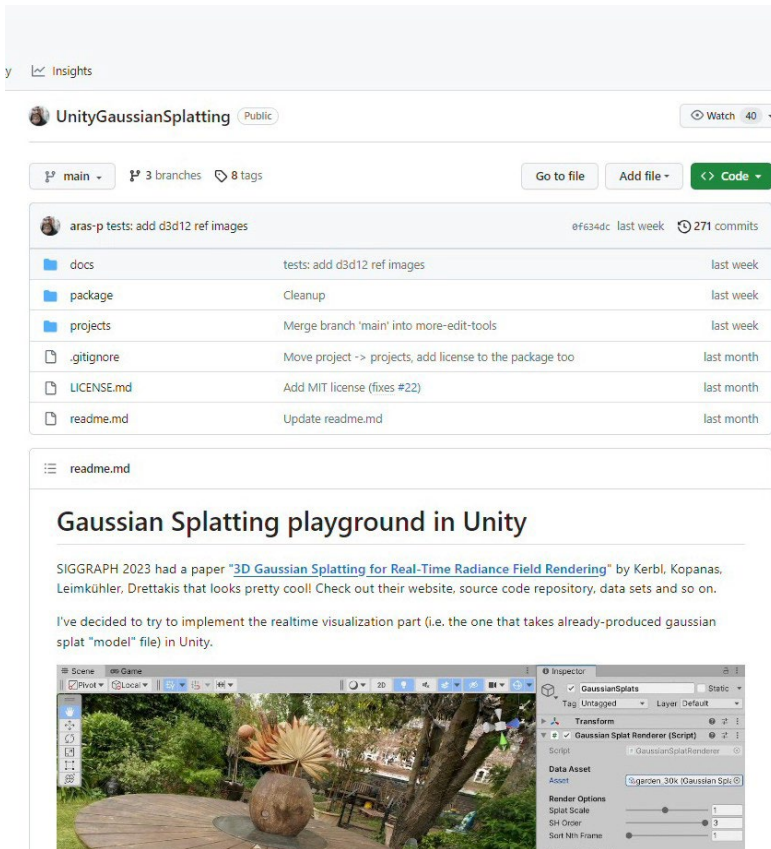
Long Term:

1. Robust, efficient and dynamic 3D reconstruction

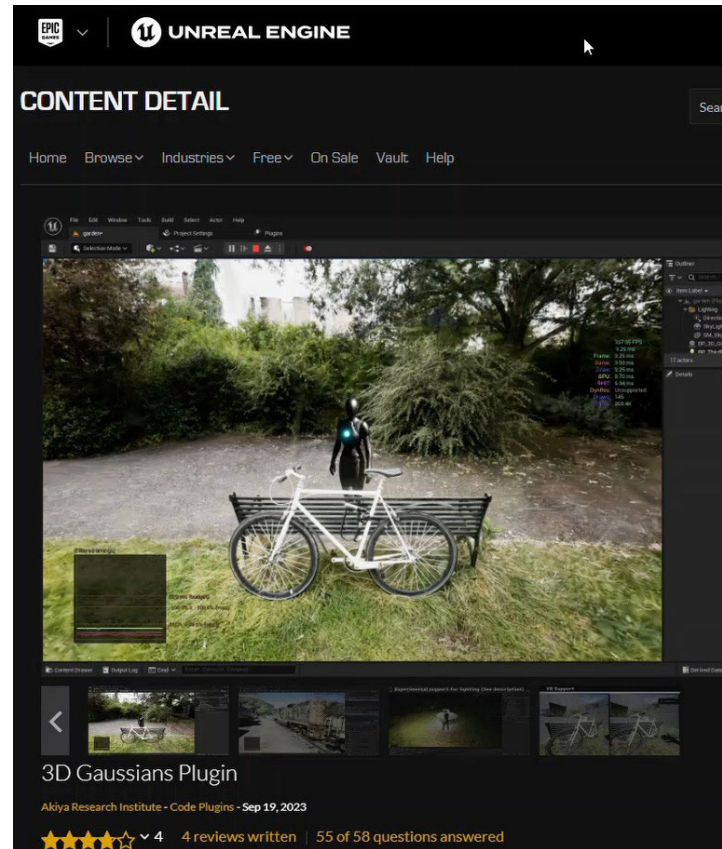
Short Term:

1. Vfx
2. Retail – E-commerce
3. 3D Grounded Video Editing

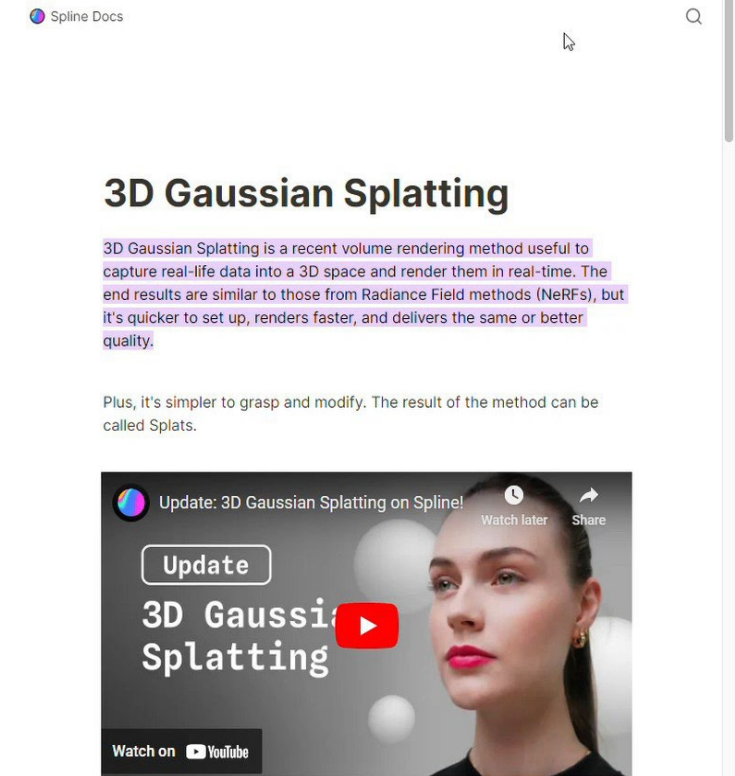
Gaussian Splatting in Graphics Engines



Gaussian Splatting in Unity



Gaussian Splatting in Unreal



How to create and import Splats (Drag

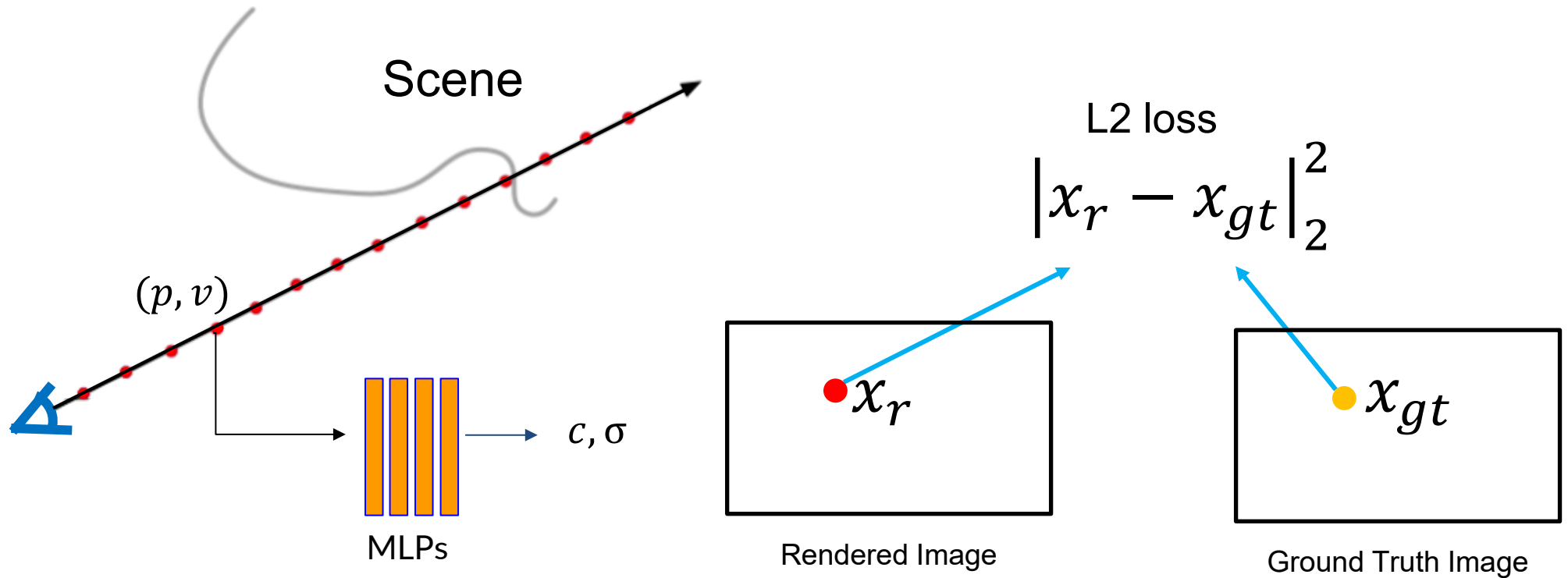
Gaussian Splatting in Spline

Limitations

1. Handcrafted heuristics for densification.
2. Popping artifacts because of the mean-based sorting.
3. Representation Size
 - a. 3DGS: 350 - 700MB (3-6m of Gaussians)
 - b. INGP: 15 - 50MB
 - c. MipNeRF360: 8.6MB

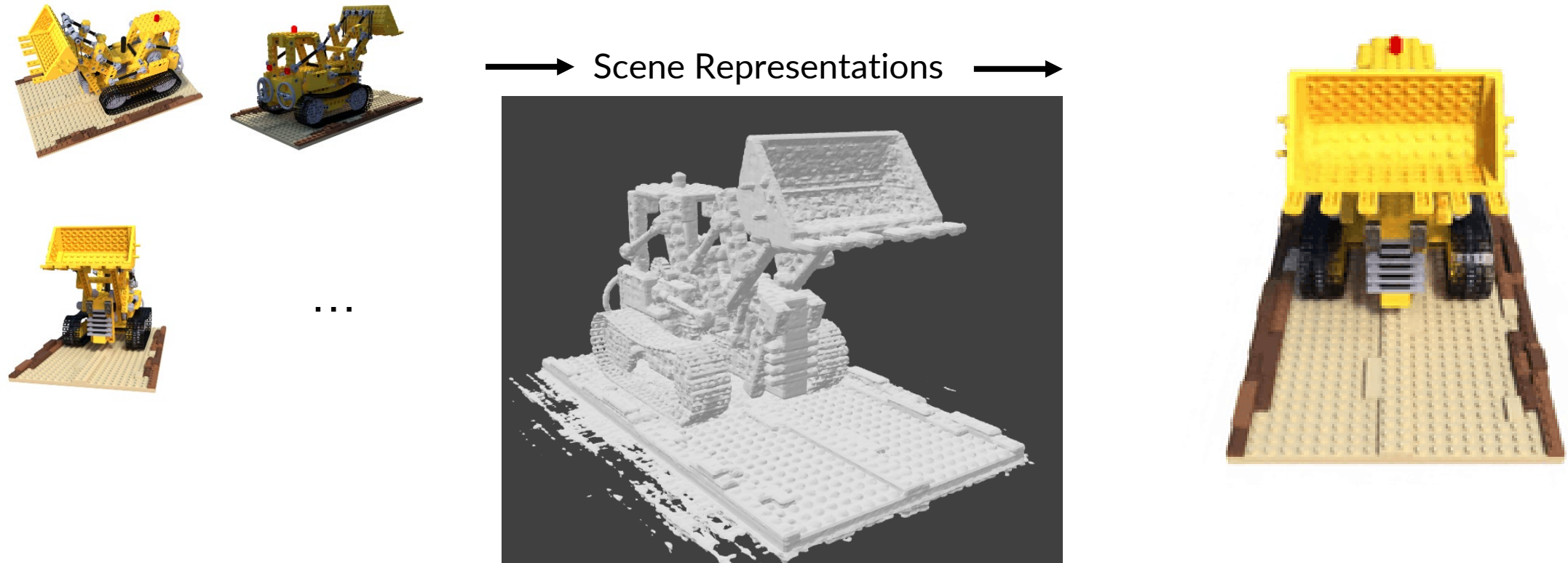
Neural SDF Representation Learning

Neural Radiance Fields (NeRF)



[Mildenhall et al. 2020]

Surfaces Extracted from Learned Representation

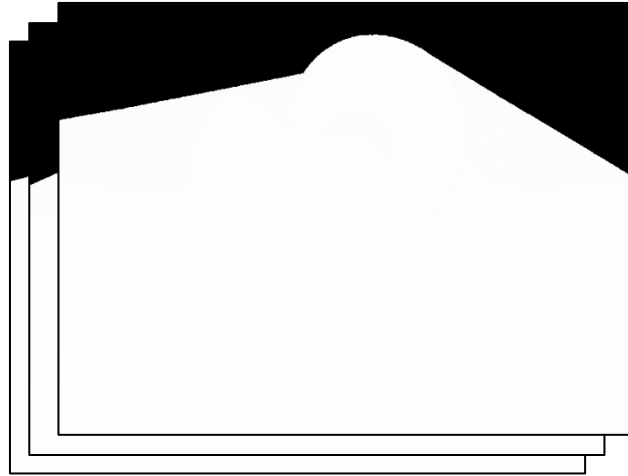


Volume density used as scene representation lacks surface constraints

Background - Surface Reconstruction Methods



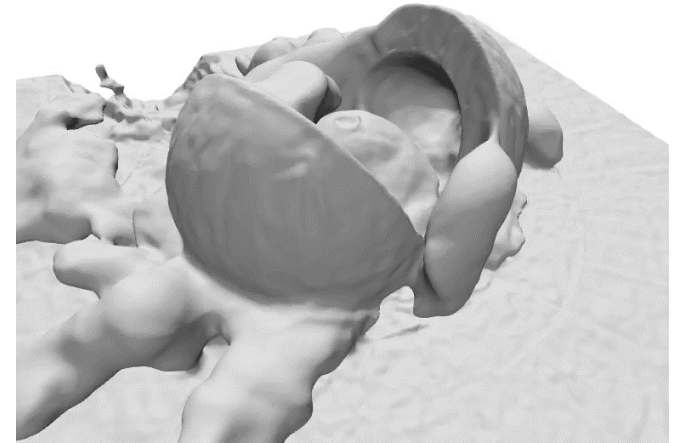
Images



Masks



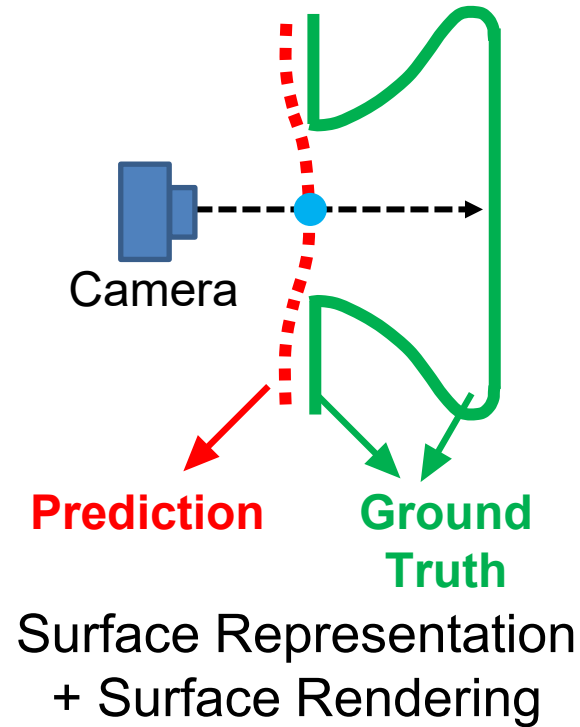
Supervise
Signed
Distance Fields
(SDF)



Geometry
(Represented as SDF)

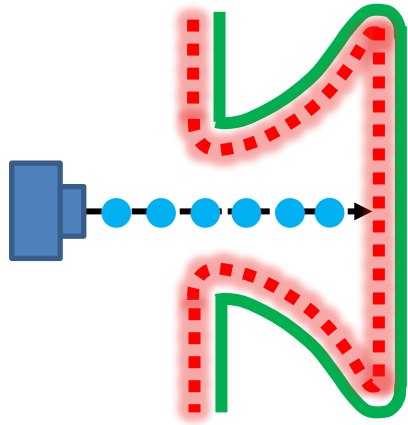
IDR [Yariv et al. 2020]

Background - Surface Reconstruction Methods

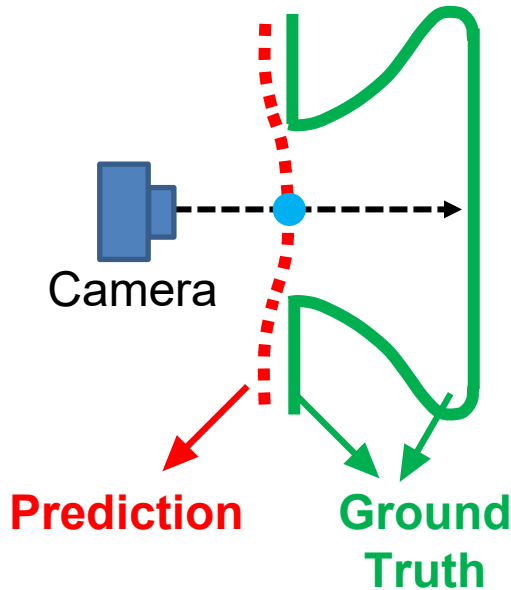


Surface rendering is not suitable for learning scene representation

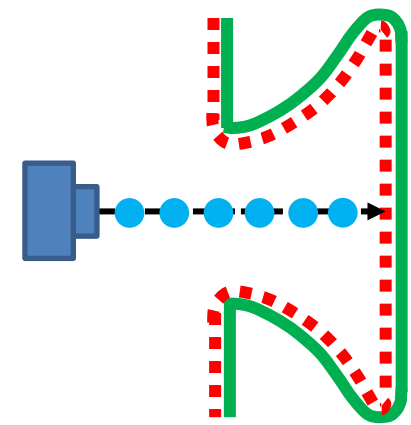
Surface Representation + Volume Rendering



Volume Representation
+ Volume Rendering



Surface Representation
+ Surface Rendering



Surface Representation
+ Volume Rendering

P. Wang, **L. Liu**, Y. Liu, C. Theobalt, T. Komura, W. Wang. NeuS: Learning Neural Implicit Surfaces by Volume Rendering for Multi-view Reconstruction, NeurIPS 2021 Spotlight

Challenge

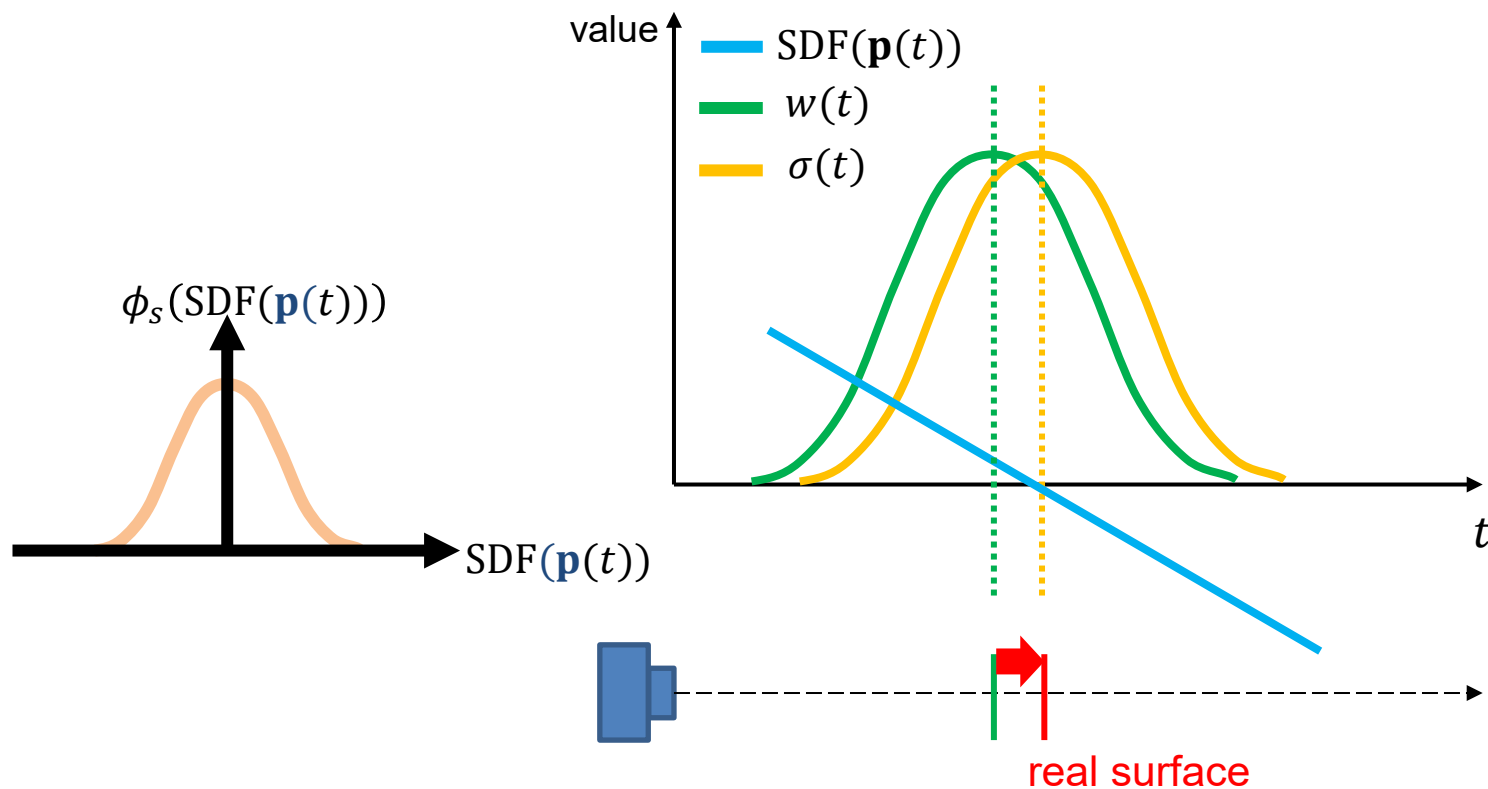
Simply applying volume rendering to the density associated with SDF.

$$\text{➤ } \mathcal{C} = \int_0^{+\infty} w(t)c(t)dt$$

$$\text{➤ } w(t) = T(t)\sigma(t)$$

$$\text{➤ } T(t) = e^{-\int_0^t \sigma(u)du}$$

$$\text{➤ } \sigma(t) = \phi_s(\text{SDF}(\mathbf{p}(t)))$$



Problem: Introduce bias, i.e., the maximal weight term $w(t)$ is not attained on the surface intersection.

Our Solution

Requirements on the weight function:

Unbiased: $w(t)$ attains a locally maximal value at a surface intersection point $\mathbf{p}(t^*)$, i.e. with $f(\mathbf{p}(t^*)) = 0$

Occlusion-aware: Given any two depth values t_0 and t_1 satisfying $f(\mathbf{p}(t_0)) = f(\mathbf{p}(t_1))$, $w(t_0) > 0$, $w(t_1) > 0$, and $t_0 < t_1$, there is $w(t_0) > w(t_1)$.

Our Solution

We first introduce a straightforward way to construct an unbiased weight function

$$w(t) = \frac{\phi_s(f(\mathbf{p}(t)))}{\int_0^{+\infty} \phi_s(f(\mathbf{p}(u))) du}$$

where $\phi_s(x) = se^{-sx}/(1 + e^{-sx})^2$,
 $f(\mathbf{p}(t))$ is the SDF value of point $\mathbf{p}(t)$

However, this weight function is not occlusion-aware.

Our Solution

We design a weight function that is both occlusion-aware and unbiased in the first order approximation of SDF by combining the following two equations.

Occlusion-aware but ***NOT*** unbiased

- $w(t) = T(t)\sigma(t)$
- $T(t) = e^{-\int_0^t \sigma(u)du}$
- $\sigma(t) = \phi(f(\mathbf{p}(t)))$

Unbiased but ***NOT*** occlusion-aware

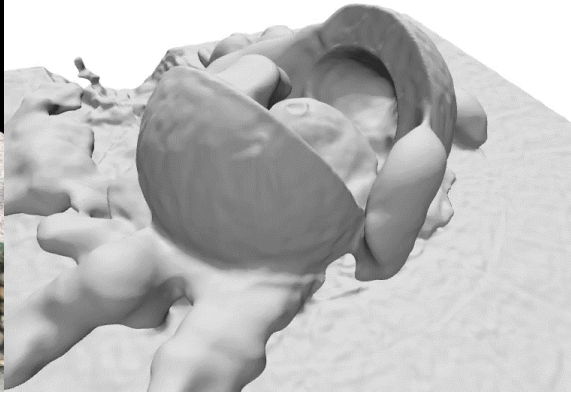
$$w(t) = \frac{\phi_s(f(\mathbf{p}(t)))}{\int_0^{+\infty} \phi_s(f(\mathbf{p}(u))) du}$$

where $\phi_s(x) = se^{-sx}/(1 + e^{-sx})^2$,
 $f(\mathbf{p}(t))$ is the SDF value of point $\mathbf{p}(t)$

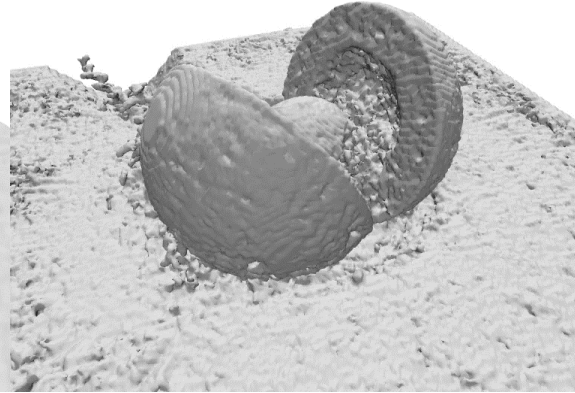
Comparison



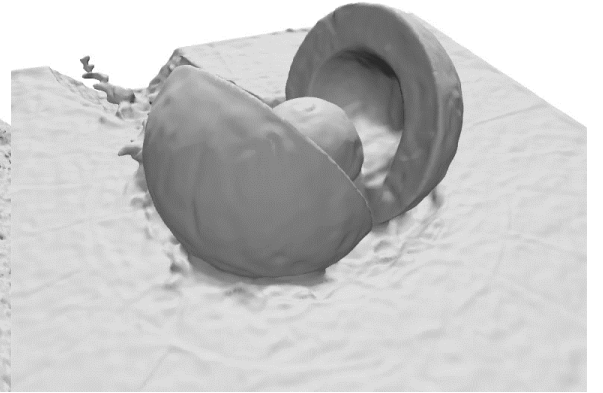
Reference Image



IDR
[Yariv et al. 2020]

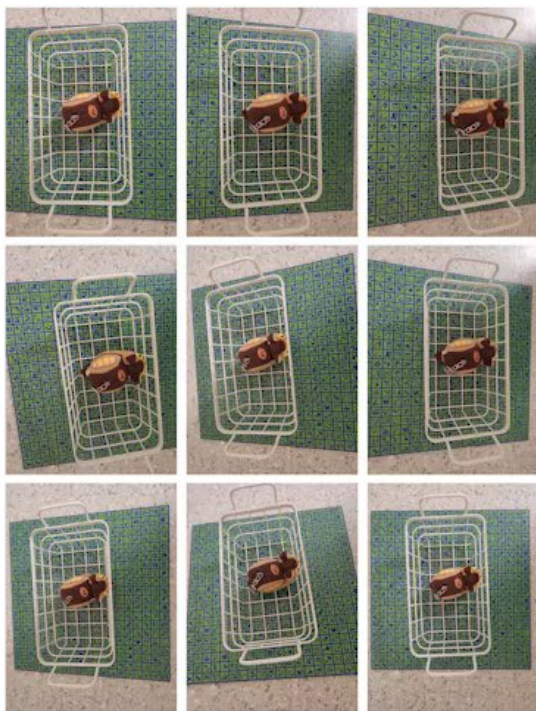


NeRF
[Mildenhall et al. 2020]

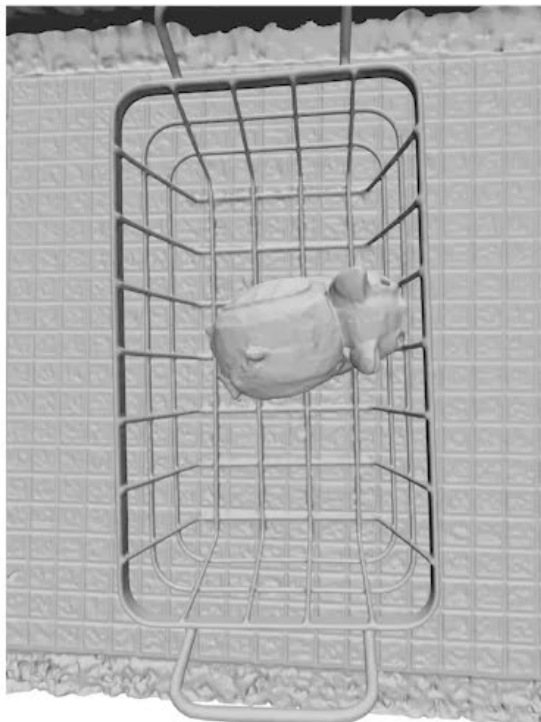


NeuS
(Ours)

Results of NeuS



A subset of input images

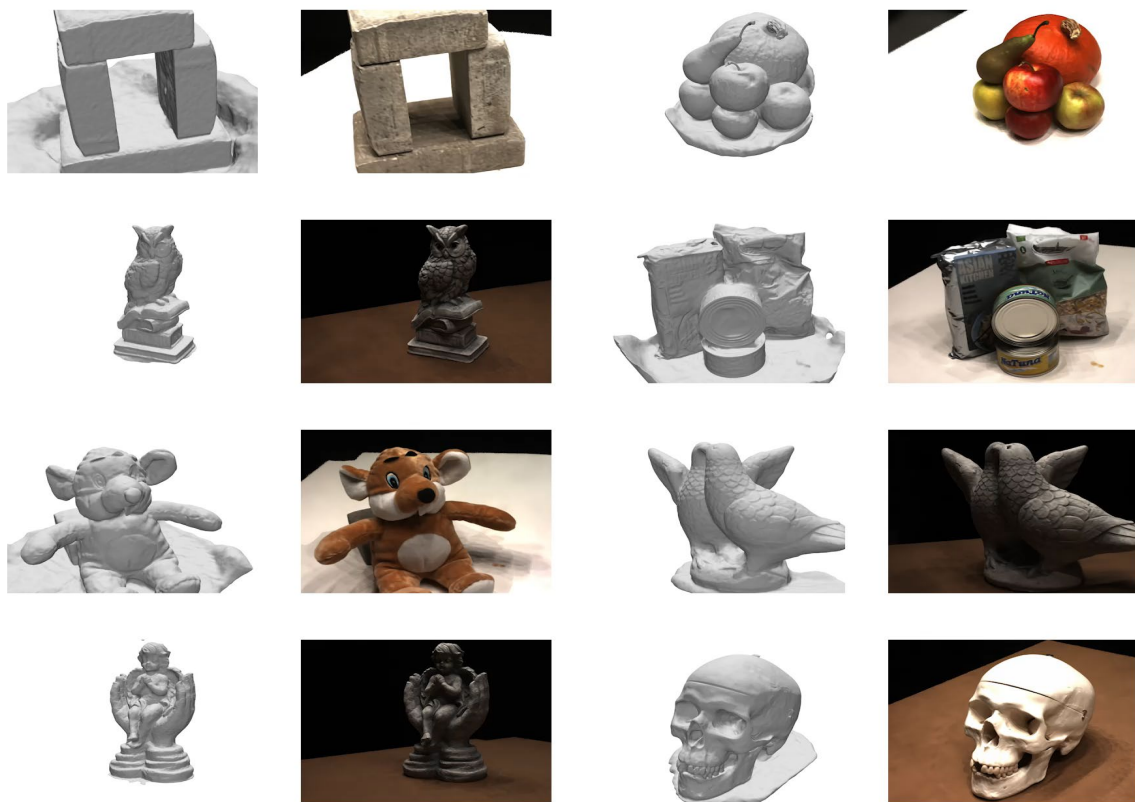


Our surface geometry
(w/o mask supervision)

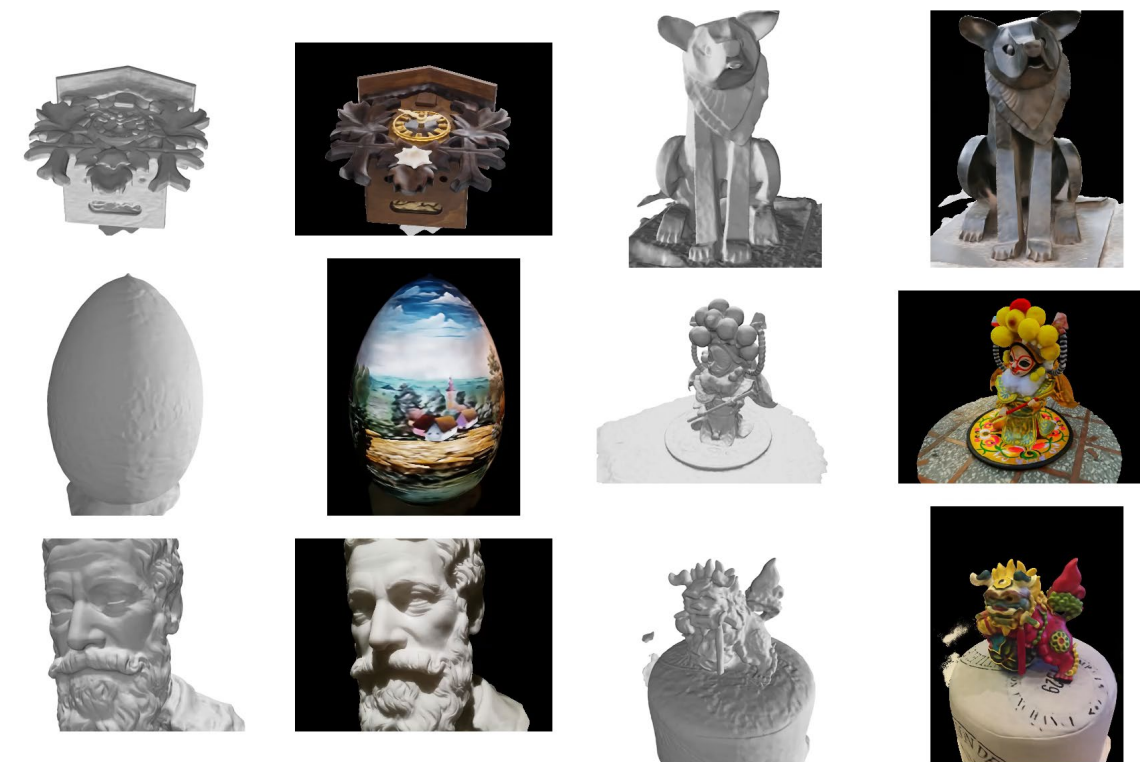


Our rendering
(w/o mask supervision)

Results of NeuS



Geometry Rendering Geometry Rendering
DTU dataset

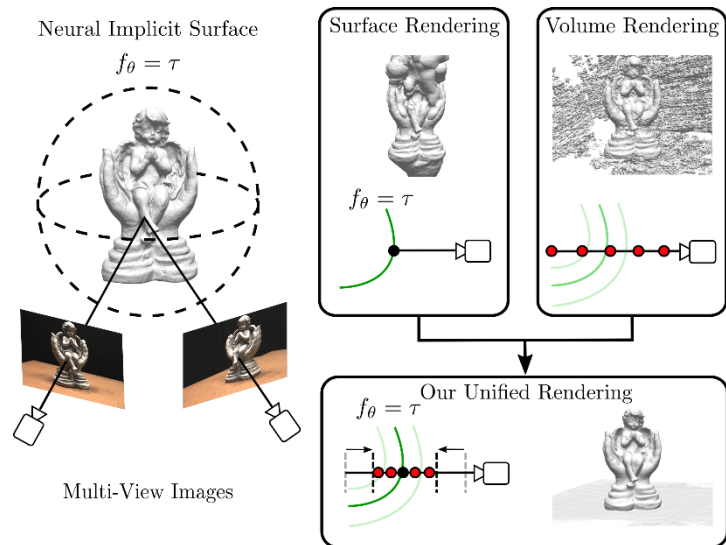


Geometry Rendering Geometry Rendering
BlendedMVS dataset

Concurrent Work



Yariv et al. 2021. Volume Rendering of Neural Implicit Surfaces, NeurIPS 2021 Oral



Oechsle et al. 2021. UNISURF: Unifying Neural Implicit Surfaces and Radiance Fields for Multi-View Reconstruction, ICCV 2021 Oral

Fast Training of NeuS



NeuS



NeuS2

minutes seconds

00:01

Y. Wang*, Q. Han*, M. Habermann, K. Daniilidis, C. Theobalt, **L. Liu**. NeuS2: Fast Learning of Neural Implicit Surfaces for Multi-view Reconstruction. ICCV 2023.

Fast Training of NeuS



Dynamic Scenes (20 seconds per frame)

Neural SDF -> UDF



Input images



Reconstructed
GT mesh



Ours



NeuS



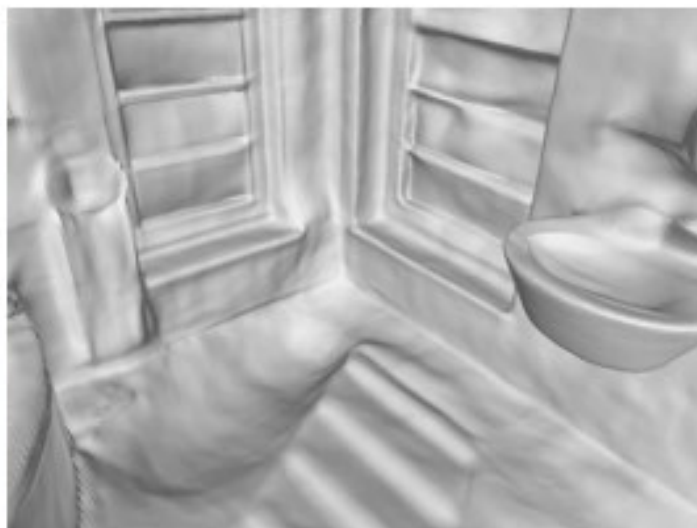
VolSDF

X. Long, C. Lin, **L. Liu**, Y. Liu, P. Wang, C. Theobalt, T. Komura, W. Wang. NeuralUDF: Learning Unsigned Distance Fields for Multi-view Reconstruction of Surfaces with Arbitrary Topologies. CVPR 2023.

Indoor Scene Reconstruction



Reference



NeuS



NeuRIS

J. Wang, P. Wang, X. Long, C. Theobalt, T. Komura, **L. Liu**, W. Wang. NeuRIS: Neural Reconstruction of Indoor Scenes Using Normal Priors, ECCV 2022